



GitHub Trending Top 10

每日热门开源项目深度解读

2026-09-03

今日榜单

1	fmtlib/fmt A modern formatting library	C++	2天
2	mattpocock/skills Skills for Real Engineers. Straight from my .agents directory.	Shell	NEW
3	NousResearch/hermes-agent The agent that grows with you	Python	2天
4	DietrichGebert/ponytail Makes your AI agent think like the laziest senior dev in the room. The best code...	JavaScript	2天
5	anthropics/skills Public repository for Agent Skills	Python	NEW
6	affaan-m/ECC The agent harness performance optimization system. Skills, instincts, memory, se...	JavaScript	NEW
7	JuliusBrussee/caveman why use many token when few token do trick — Claude Code skill that cuts 65% o...	Go	NEW
8	blader/humanizer Agent skill that removes signs of AI-generated writing from text	Python	NEW
9	google-research/timesfm TimesFM (Time Series Foundation Model) is a pretrained time-series foundation mo...	Python	2天
10	averygan/reclip Download videos from almost any website. Lightweight, self-hosted media download...	HTML	NEW

#1

C++

连续 2 天

fmt

A modern formatting library

Stars **24,786** Forks **3,003** Today **+955**

<https://github.com/fmtlib/fmt>

项目简介：{fmt} 是一个开源的 C++ 格式化库，旨在提供比 C 的 `stdio` 和 C++ 的 `iostreams` 更快速、更安全的替代方案。它解决了传统格式化方法中类型不安全、容易出错以及性能不佳等问题，是 C++20 标准中 `std::format` 的参考实现。

核心功能：最核心的功能是提供了类似 Python 的简洁格式化语法，支持位置参数和类型安全的格式化操作。其次，它实现了 C++20/23 标准中的 `std::format` 和 `std::print`，并具备基于 Dragonbox 算法的高性能浮点数格式化能力。此外，它还支持用户自定义类型的格式化扩展和编译期格式字符串检查。

适用场景：任何需要高性能、类型安全字符串格式化的 C++ 项目都可以使用，尤其是对输出性能有严格要求的底层库或应用。它也适合需要国际化支持（通过位置参数）或希望统一格式化代码风格的大型项目，同时可以作为尚未完全支持 C++20 格式化的旧编译器环境下的替代方案。

技术亮点：其技术核心在于对浮点数格式化的深度优化，采用 Dragonbox 算法实现正确舍入且性能远超标准库实现。另一个亮点是极佳的可移植性和轻量性，最小配置仅需三个头文件，无外部依赖，且支持纯头文件模式。此外，它实现了编译期格式字符串检查，能提前发现错误，从根源上避免运行时崩溃。

上手建议：对于使用者，建议直接阅读官方文档中的“快速入门”部分，并在 Compiler Explorer 上尝试示例代码，以最快感受其 API 的简洁性。对于想贡献代码的开发者，可以从查看 GitHub 仓库的 CONTRIBUTING 指南和标记为“good first issue”的议题开始，项目对新手友好，测试体系完善。

#2

Shell

NEW

skills

Skills for Real Engineers. Straight from my .agents directory.

Stars **246,383** Forks **20,910** Today **+1,576**

<https://github.com/mattpocock/skills>

项目简介：这是前端知名教育家 Matt Pocock 开源的一套“AI 编程智能体技能包”，本质上是将真实工程经验沉淀为结构化的指令文件，让 Claude Code、Codex 等编码代理在开发时遵循更严谨的工程流程。项目反对“vibe coding”（凭感觉写代码）的浮躁方式，旨在让 AI 辅助开发回归可控、可复现的工程实践。

核心功能：最亮眼的是“盘问式需求澄清”（/grill-me 与 /grill-with-docs），通过让 AI 先提问来对齐需求，避免开发方向跑偏。其次，它提供一套可组合的小型技能文件，覆盖从需求分析到编码执行的完整流程。此外，项目内置了与 GitHub/Linear 等主流 Issue 追踪器的集成能力，并能按用户偏好自动打标签和归档文档。

适用场景：主要面向使用 Claude Code、Codex 等 AI 编程工具的软件工程师，尤其是那些对代码质量有要求、希望 AI 能遵循真实工程规范而非只做代码补全的开发者。无论是个人项目还是团队协作，只要你在用 AI 写代码并苦于“答非所问”或“过度啰嗦”，这套技能包都能派上用场。

技术亮点：项目设计上刻意避开 GSD、BMAD 等“重型框架”对流程的垄断，转而采用“小而美、可组合、易修改”的哲学，让每个技能文件都是独立的 Markdown 指令，既透明又便于用户按需调整。它还提供了双轨分发机制：Claude Code 插件作为只读托管包自动更新，而 skills.sh 则允许将文件复制进项目自由修改，兼顾了“订阅”与“定制的自由”。

上手建议：安装过程非常轻量，30 秒即可完成。Claude Code 用户直接运行 `claude plugins install mattpocock-skills`，其他代理（如 Codex）用 `npx skills@latest add mattpocock/skills` 选择所需技能。装完后在仓库中执行一次 `/setup-matt-pocock-skills` 完成个性化配置，即可体验。想贡献的话，可以从阅读 `.agents/adr/` 下的架构决策记录开始，理解其设计取舍后再提 PR。

#3

Python

连续 2 天

hermes-agent

The agent that grows with you

Stars **240,574** Forks **49,253** Today **+778**

<https://github.com/NousResearch/hermes-agent>

项目简介：Hermes Agent 是由 Nous Research 推出的自我进化型 AI 智能体，核心卖点是“随你成长”——它能在使用中不断学习、沉淀技能和记忆，逐步构建对用户的深度理解。它解决的是传统 AI 助手“每次对话都从零开始”的痛点，让智能体真正拥有跨会话的连续性和成长性。

核心功能：一是内置“闭环学习回路”，能自主创建技能、在使用中优化技能，并主动记忆和检索过往对话；二是支持多平台接入（Telegram、Discord、Slack 等）和跨平台会话延续；三是提供真实的终端 TUI 界面，支持多行编辑、命令补全和流式工具输出；四是内置定时任务调度器，可用自然语言设定每日报告、夜间备份等自动化流程；五是支持子代理并行委派和多种远程执行后端（Docker、SSH、无服务器等）。

适用场景：适合需要长期、持续性 AI 辅助的开发者或研究者——比如在云端 VPS 上部署一个“永不掉线”的助手，通过 Telegram 随时指挥它干活；也适合需要批量生成训练轨迹的 AI 研究团队，或想要自动化日常运维（备份、审计、报告）的个人用户。对于在意数据主权、不想被绑定在单一模型或平台上的用户尤其友好。

技术亮点：最值得关注的是它的“学习回路”设计——技能以开放标准（agentskills.io）存储，可跨会话自我改进；记忆系统结合 FTS5 全文搜索与 LLM 摘要，实现高效的跨会话召回。其次是它的“随处运行”架构：支持七种终端后端，其中 Modal 和 Daytona 提供无服务器持久化，环境空闲时休眠、唤醒时恢复，成本近乎为零。此外，模型提供商可切换（hermes model 命令），无锁定风险。

上手建议：新用户只需一条 curl 命令即可在 Linux/macOS/WSL2 上完成安装，Windows 原生也有 PowerShell 脚本支持。想深度使用可从官方文档（hermes-agent.nousresearch.com/docs）入门，先尝试在 Telegram 上连接并让它执行一个简单任务，观察它的技能创建过程。想贡献代码的开发者可关注 GitHub Issues 中的 bug 反馈和功能请求，项目采用 MIT 协议，社区活跃于 Discord 频道。

#4

JavaScript

连续 2 天

ponytail

Makes your AI agent think like the laziest senior dev in the room. The best code is the code you never wrote.

Stars **122,881** Forks **6,640** Today **+2,138**

<https://github.com/DietrichGebert/ponytail>

好的，这是对 DietrichGebert/ponytail 项目的解读：

项目简介：Ponytail 是一个为 AI 编程助手（如 Claude Code）注入“资深懒散工程师”思维模式的技能包。它通过一套精心设计的提示词，引导 AI 在编码时优先寻找最简单、最直接的解决方案，力求用最少的代码完成任务，核心理念是“最好的代码是你从未写过的代码”。

核心功能：1. 通过“技能”让 AI 遵循极简主义，避免过度设计；2. 内置多个实际案例，教 AI 识别并避免常见的“过度构建”陷阱（如用原生 `<input type="date">` 替代复杂的日期选择器组件）；3. 提供一套可复现的基准测试框架，用于量化代码精简带来的效率提升。

适用场景：主要面向使用 AI 编程助手进行日常开发的软件工程师和技术团队。当 AI 生成的代码常常“杀鸡用牛刀”时，Ponytail 能显著减少代码量、缩短处理时间并降低成本，尤其适合需求明确、追求代码简洁和可维护性的项目。

技术亮点：项目提供了令人信服的量化基准测试数据。在真实仓库的测试中，使用后代码量平均减少 54%（最高达 94%），时间缩短 27%，成本降低 20%，且安全性保持 100%。这表明其效果不只是“风格偏好”，而是有实际数据支撑的效率优化。

上手建议：如果你是 AI 编程工具的用户，可以查看其 README 中的安装说明将其作为技能添加到你的 AI 配置中。如果你想深入了解或贡献，可以从 benchmarks/ 目录入手，查看或复现其测试方法，或研究 examples/ 中的案例来学习如何编写有效的极简指令。

#5

Python

NEW

skills

Public repository for Agent Skills

Stars **173,460** Forks **20,591** Today **+277**

<https://github.com/anthropics/skills>

项目简介：这是 Anthropic 官方发布的 Agent Skills（智能体技能）公共仓库，核心是让 Claude 能够按需加载一组结构化的指令、脚本和资源，从而在特定任务上表现得更专业。它解决的是大模型在通用能力之外，如何高效复用和标准化执行特定工作流的问题。

核心功能：1) 提供了一套完整的 Agent Skills 规范（spec），定义了技能的标准格式；2) 内置了丰富的示例技能库，涵盖创意设计、技术开发、企业办公和文档处理等多个领域；3) 开放了驱动 Claude 文档能力的 docx/pdf/pptx/xlsx 技能源码作为复杂技能参考；4) 支持通过 Claude Code 插件市场一键安装，也可在 Claude.ai 和 API 中直接使用。

适用场景：开发者想要为 Claude 构建自定义技能或学习最佳实践时，可以参考这里的示例和模板；企业用户需要让 Claude 按照公司品牌规范生成文档、按特定流程分析数据时，可直接安装对应技能；任何希望扩展 Claude 在垂直领域能力的个人或团队都能从中受益。

技术亮点：技能被设计为自包含的文件夹结构，以 SKILL.md 文件为核心，通过 YAML frontmatter 声明技能的元数据（名称和描述），正文则用 Markdown 编写具体指令。这种极简设计大幅降低了技能创建的准入门槛，同时保持了良好的可移植性和版本控制友好性。

上手建议：建议从阅读 template 目录的模板开始，理解 SKILL.md 的基本结构；然后浏览 skills 目录下的示例，挑选与自己业务相近的技能研究其实现模式；如果想快速体验，可以在 Claude Code 中执行 `/plugin marketplace add anthropics/skills` 安装示例插件，直接试用文档处理技能。

#6

JavaScript

NEW

ECC

The agent harness performance optimization system. Skills, instincts, memory, security, and research-first development for Claude Code, Codex, Opencode, Cursor and beyond.

Stars **246,763** Forks **37,185** Today **+749**

<https://github.com/affaan-m/ECC>

项目简介

ECC（Agent Harness 性能优化系统）是一个为 AI 编程助手（如 Claude Code、Codex 等）提供系统级增强的 JavaScript 项目，定位类似于“智能体操作系统”。它解决的核心问题是：原生 AI 编程助手缺乏长期记忆、安全防护、技能扩展和性能优化的能力，导致在复杂项目中难以持续高效工作。

核心功能

- **技能（Skills）与本能（Instincts）系统**：为 AI 智能体预置和注入可复用的行为模式与工作流程，使其在特定任务上表现更专业。
- **持久化记忆**：让 AI 智能体在多次会话中记住项目上下文、用户偏好和历史决策，避免每次都从零开始。
- **安全防护（AgentShield）**：提供安全层来约束 AI 智能体的操作边界，防止其执行危险命令或访问敏感信息。
- **性能优化**：针对不同 Agent Harness 进行系统级调优，提升响应速度和任务执行效率。
- **跨平台兼容**：不仅支持 Claude Code，还兼容 Codex、Opencode、Cursor 等主流 AI 编程工具，通过统一的 ecc-universal 包实现一次配置、多处使用。

适用场景

主要面向重度使用 AI 编程助手的开发者和团队，尤其是那些在大型代码库、长期项目中依赖 AI 协作的用户。典型场景包括：需要 AI 智能体保持项目记忆的跨会话开发、需要严格安全管控的企业环境、以及希望将 AI 编程助手定制为团队专属“数字员工”的技术团队。

技术亮点

项目采用多语言架构（Shell + TypeScript + Python + Go + Java + Perl），通过 npm 包和 GitHub App 双通道分发，实现了插件化的“hook profile”机制来深度嵌入不同 AI 工具。值得关注的是其官方渠道验证体系——专门警告用户防范第三方恶意重打包，说明项目对供应链安全有较高敏感度。此外，API 网关（api.ecc.tools）实时统计安装量和星标数据，形成了完整的生态数据闭环。

上手建议

如果你是使用者，最直接的路径是从终端运行 `npx ecc-universal setup` 进行引导式安装，前提是已安装 Node.js 18+、Git 和 Claude Code 2.1+。如果你是贡献者，建议先浏览项目官网 ecc.tools 了解架构文档，再通过 Discord 社区与维护者沟通，关注 GitHub 仓库中的 issue 和贡献指南。注意务必只从官方渠道获取代码和安装包，避免使用任何第三方镜像或转载。

#7

Go

NEW

caveman

why use many token when few token do trick — Claude Code skill that cuts 65% of tokens by talking like caveman

Stars **102,920** Forks **5,985** Today **+545**

<https://github.com/JuliusBrussee/caveman>

项目简介：这是一个颇具黑色幽默的 AI 工具项目，核心思路是“用最少的 token 说清事”。它通过让 Claude 等 AI 代理用“原始人语”回复，砍掉客套话和冗余修饰，从而节省高达 65% 的 token 费用。简单说，就是在不牺牲答案质量的前提下，让 AI 说话更“抠门”。

核心功能： 1. **Caveman Skill：**一个规则文件，可让 30+ 种 AI 代理（Claude Code、Codex、Gemini 等）自动切换为极简回复风格，一条命令即可安装。 2. **Caveman Proxy：**本地代理层，在请求发给 AI 前先压缩上下文，且保留原始副本供代理回溯，进一步降低输入 token。 3. **双轨叠加：**Skill 负责“输出精简”，Proxy 负责“输入压缩”，两者可叠加使用，实现双向省钱。 4. **多代理适配：**原生支持 10 种主流代理的封装，并兼容 30+ 种基于 Skill 的代理。

适用场景：重度依赖 AI 编程助手或 API 的开发者、团队，尤其是每天产生数百次 AI 交互、token 账单较高的用户。也适合在长会话或复杂项目中，需要控制上下文窗口成本的使用者。

技术亮点：项目设计很巧妙——它精准识别“哪些内容不能压缩”（代码、路径、错误信息）与“哪些内容可以精简”（解释性散文），确保技术准确性的同时最大化压缩率。Proxy 的“压缩但留存副本”机制也解决了信息丢失的隐患，技术上兼顾了成本与可靠性。

上手建议：从轻量级 Skill 开始，一条 `npx skills add JuliusBrussee/caveman` 命令即可体验效果。若觉得 token 节省还不够明显，再升级到 Proxy 模式。想贡献代码的话，可关注其 Go 语言实现的压缩逻辑或代理封装层。

#8

Python

NEW

humanizer

Agent skill that removes signs of AI-generated writing from text

Stars **41,123** Forks **3,533** Today **+1,214**

<https://github.com/blader/humanizer>

项目简介：

Humanizer 是一个基于 Python 的“AI 文本去味”工具，本质上是一个 Agent skill（技能包），专门把 ChatGPT、Claude 等生成的文章改写得像真人手写。它不改变原文含义，只消除那些一眼就能看出的“AI 腔”。

核心功能：

- 内置 35 种“AI 写作痕迹”识别模式（源自维基百科 WikiProject AI Cleanup 的总结），覆盖措辞、语法、风格三大类。
- 支持直接粘贴文本、自然语言指令，或指定文件路径进行改写，可保留代码、数据、链接等非散文内容。
- 提供“两遍式”流程：先重写，再对照原始事实与 AI 模式清单自查，输出最终稿和一份“哪里还像 AI”的简要批注。
- 支持“声音匹配”：提供 2-3 段你自己的写作样本，它会模仿你的节奏、用词和标点习惯。

适用场景：

内容创作者、编辑、学生和开发者，凡是需要把 AI 生成的草稿变成“人味”文章的场景都适用。尤其适合批量改写博客、营销文案、学术初稿，以及任何需要规避 AI 检测器或让读者觉得“是真人写的”的场景。因为它以 Markdown 技能形式存在，可以嵌入任何支持 skills 的 AI Agent 工作流。

技术亮点：

最值得关注的是它没有用“重写模型”的暴力方案，而是基于维基百科人工维护的 35 种反模式清单做规则驱动地改写 + 自检。这种做法的好处是**可解释、可验证**：它不会凭空捏造事实（名字、数字、日期必须有来源），且每次输出都会展示“第一次改写稿”和“批注”，让用户看到它改了什么、为什么改。另外，它区分“个人写作”（保留风格）和“技术文档”（保持中性），体现了对语域（register）的敏感。

上手建议：

想用的人直接在你的 Agent（如 Claude、Cursor 等支持 skills 的环境）里调用 /humanizer 命令并粘

贴文本即可，零安装成本。想贡献的话，可以从阅读 README 中那张 35 种模式的表格开始——项目最核心的“知识库”就是这张表，任何新增的 AI 写作痕迹模式都是高价值 PR。另外，项目用纯 Markdown 编写，逻辑透明，适合作为学习“如何给 Agent 设计可审计的文本处理流程”的参考案例。

#9

Python

连续 2 天

timesfm

TimesFM (Time Series Foundation Model) is a pretrained time-series foundation model developed by Google Research for time-series forecasting.

Stars **30,579** Forks **2,916** Today **+1,626**

<https://github.com/google-research/timesfm>

项目简介：TimesFM 是谷歌研究院开发的时间序列基础模型，专注于时间序列预测任务。它采用类似大语言模型的预训练范式，让用户无需针对特定任务训练即可进行高质量的时序预测，解决了传统时序模型需要大量领域数据和训练时间的问题。

核心功能：一是原生支持多变量时间序列预测，可同时处理多通道数据；二是支持灵活的协变量输入，包括仅过去和过去加未来的动态协变量；三是零样本泛化能力出色，在 fev-bench、TIME 和 GIFT-Eval 三大基准上均排名第一；四是支持超长上下文（最多 16k）和连续分位数预测。

适用场景：适合需要快速进行时序预测的开发者、数据科学家和业务分析师，典型场景包括零售销量预测、金融指标分析、能源需求预估、运维监控告警等。尤其适合那些缺乏充足历史数据和算力资源、但需要快速获得可靠预测结果的团队。

技术亮点：TimesFM 采用 decoder-only 架构，将时序预测问题类比为语言建模中的下一个词预测，这一思路在 ICML 2024 发表。模型通过预训练学习通用时序模式，在零样本条件下即可超越许多专门训练的模型。此外，TimesFM 2.5 将参数量从 500M 压缩至 200M，同时提升了上下文长度和推理效率，体现了模型迭代的工程优化能力。

上手建议：新用户可通过 PyPI 安装 timesfm 包，加载 Hugging Face 上的预训练权重快速体验预测功能。建议先阅读 timesfm-forecasting/ 目录下的文档和示例代码，了解基本 API 调用方式。若需深入使用，可参考 LoRA 微调示例进行领域适配；想贡献代码的开发者可从 AGENTS.md 和 SKILL.md 入手了解项目规范。注意 TimesFM 3.0 权重目前仅限非商业用途，商业应用需留意许可证限制。

#10

HTML

NEW

reclip

Download videos from almost any website. Lightweight, self-hosted media downloader with a clean web UI.

Stars **8,199** Forks **1,355** Today **+673**

<https://github.com/averygan/reclip>

项目简介：ReClip 是一个自托管的开源媒体下载工具，通过简洁的 Web 界面，让用户可以从 YouTube、TikTok、Instagram 等 1000+ 网站下载视频或提取音频。它解决了"想存离线内容但各平台下载方式不一"的痛点，把复杂的下载流程统一封装成了一个开箱即用的本地服务。

核心功能：一是支持 1000+ 网站的链接解析和下载（底层由 yt-dlp 驱动）；二是提供 MP4 视频与 MP3 音频两种输出格式，并支持清晰度选择；三是支持批量粘贴多个 URL 并自动去重，一键下载全部；四是采用纯 HTML/CSS/JS 的无框架前端，界面干净且响应式，无需任何构建步骤。

适用场景：适合需要离线收藏教学视频、保存社交媒体上的灵感素材、提取播客或音乐音频的个人用户。也适合极客或小型团队在家庭服务器或 NAS 上自建一个"媒体下载站"，供多设备通过浏览器随时使用。

技术亮点：后端仅约 150 行 Python (Flask) 代码，依赖精简到只有 2 个库，却借助 yt-dlp 获得了覆盖千余网站的解析能力，体现了"小壳大核"的极简设计哲学。前端零框架、零构建，单文件搞定交互，配合自动去重和批量下载逻辑，让轻量级工具也能有顺滑的批量体验。

上手建议：使用门槛极低，本地只需 `brew install yt-dlp ffmpeg` 后运行 `./reclip.sh`，或直接用 Docker 一条命令拉起服务。想参与贡献的话，可以从 README 中的 Issues 入手——项目结构简单，前端单文件、后端单文件，非常适合作为学习 Flask + yt-dlp 集成的入门级实战项目。

数据来源: github.com/trending

报告日期: 2026-09-03

由 DeepSeek AI 提供智能分析 | 自动生成